---------- Collection in Java ----------

What are the disadvantages of array?

-> Array size is fixed in nature.
-> We can have grow able array, but it is not decided by the system when the array has to grow.
-> By default in array every cell will contain default values.
-> As the size of array is defined, if user will not use all the memory slots then there is waste of memory
-> In an array we can't store extra value, as well as we can't access any value out of the bounds.
-> Can pass only same type of data in array because array is homogeneous. Solution is make the array heterogeneous by using Object array but still the problem is there for fixed size.
-> For performing complex operations in array, we don't have predefined methods.
-> There is no guarantee of not storing duplicate values in array.
-> We can't deal with an array by following some of data structures such as LIFO,FIFO,balanced tree, sorted, non duplicates etc.

'collection' is a **container object** in which we can store multiple number of homogeneous or heterogeneous elements, duplicate elements or unique elements all together as a single unit by maintaining auto-growable feature.

In some type of collection, we can maintain the data in sorted order automatically.
In some type of collection, the data are stored but the insertion order is not maintained.

To have collection in java, we take support of interfaces and classes from **Collection Framework.**

Collection Framework means combination of classes and interfaces in java.util where we have multiple classes and interfaces relation to develop application using collection.

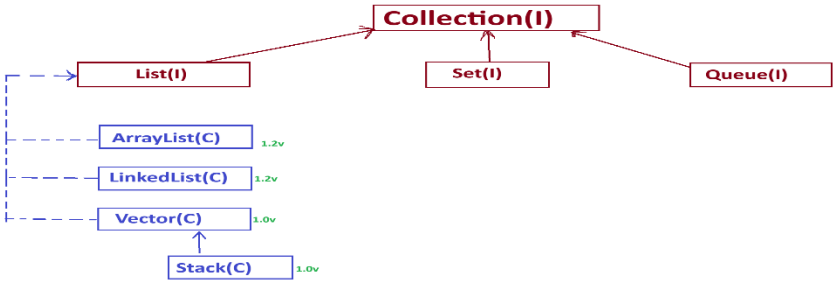
As per the real time projects, a developer requires memory of following types:

i. **A memory where duplicates are allowed**
ii. **A memory where duplicates are not allowed**
iii. **A memory for fetching the data in FIFO order.**

Based on the above memory requirement Collection has three main sub – interfaces:

1. **List**
2. **Set**
3. **Queue**

java.util



Note:

Methods in Collection(I):

1. **public abstract int size():**

It is used to return the number of elements present in container object.

e.g: /*Program to demonstrate creation of ArrayList objects with default capacity and returning size of arraylist(number of elements in arraylist).*/

```
import java.util.ArrayList;
public class TestArrayList1 {
public static void main(String[] args) {
        ArrayList a = new ArrayList();//AL object created with default capacity 10
        a.add(100);//PTV  --> IWCO---> O
        a.add(true);//PTV --> BWCO---> O
        a.add("java");// SO ---> O
        a.add(10.8);// PTV --> DWCO --> O
        System.out.println("No. of elements in container object: "+a.size());
        System.out.println(a);//toString() is invoked
}
}
```

2. **public abstract boolean isEmpty(): It is used to check whether the collection is empty or not.If the collection is empty then this method will return true else it will return false.**

```
//Program to demonstrate use of isEmpty()
import java.util.ArrayList;
public class TestIsEmpty {
public static void main(String[] args) {
        ArrayList fruit = new ArrayList();
        System.out.println("Arraylist is empty. "+fruit.isEmpty());
        System.out.println("Adding elements into arraylist");
        fruit.add("apple");
        fruit.add("grapes");
        fruit.add("orange");
        System.out.println("After adding elements, array list is empty."
        +fruit.isEmpty());
}
}
```

3. **public abstract boolean contains(java.lang.Object):**

It is used to check whether a collection contains passed object or not. If it contains the passed object then this method will return true else it will return false.

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
public class TestContainsCollection {
public static void main(String[] args) {
```

```
        //Representing array as list
        List<Integer> arr = Arrays.asList(10,20,80,50,40,30,60);
        System.out.println("40 is present. "+arr.contains(40));

}
}
```

public abstract java.util.Iterator<E> iterator(): It is used to generate Iterator cursor object on a collection.

public abstract java.lang.Object[] toArray(): It is used to return a array of Object from a collection of elements.

```
import java.util.ArrayList;
import java.util.Arrays;
public class TestToArray {
public static void main(String[] args) {
        ArrayList n = new ArrayList();
        n.add(20);
        n.add("java by Jagannath");
        n.add(8.9);
        n.add(true);
        System.out.println("Elements in collection: "+n);
        Object arr[]=n.toArray();//a new array is created
        System.out.println("Elements in array: "+Arrays.toString(arr));
}
}
```

public abstract <T> T[] toArray(T[]);

public default <T> T[] toArray(java.util.function.IntFunction<T[]>);

public abstract boolean add(E):

It is used to add element into container object.

E.g:

```
//Program to demonstrate creation of ArrayList with default capacity.
import java.util.ArrayList;
public class TestArrayList1 {
public static void main(String[] args) {
        ArrayList a = new ArrayList();//AL object created with default capacity 10
        //using add() to add element into container object
        a.add(100);//PTV  --> IWCO---> O
        a.add(true);//PTV --> BWCO---> O
        a.add("java");// SO ---> O
        a.add(10.8);// PTV --> DWCO --> O
        System.out.println(a);//toString() is invoked
}
}
```

public abstract boolean remove(java.lang.Object): It is used to remove a particular object from the collection of elements.If the element is present in the collection and we remove it then this method will return 'true' else it returns 'false'.

```
import java.util.ArrayList;
public class TestRemove {
public static void main(String[] args) {
        ArrayList<Integer> n = new ArrayList<Integer>();
        n.add(20);
        n.add(40);
```

```
        n.add(80);
        n.add(60);
        System.out.println("Elements before removing 20: "+n);
        System.out.println(n.remove(new Integer(20)));//20 is removed);
        System.out.println("Elements after removing 20: "+n);
}
}
```

public abstract boolean containsAll(java.util.Collection<?>): It is used to check whether two collections have same elements or not even if the sequence is not maintained.

```
import java.util.ArrayList;
public class TestContainsAll {
public static void main(String[] args) {
        ArrayList a = new ArrayList();
        a.add(20);
        a.add("java by Jagannath");
        a.add(8.9);
        a.add(true);
        System.out.println("Elements in a: "+a);
        ArrayList b = new ArrayList();
        b.add(20);
        b.add(8.9);
        b.add(true);
        b.add("java by Jagannath");
        System.out.println("Elements in b: "+b);
        System.out.println("b contains a: "+b.containsAll(a));
}
}
```

public abstract boolean addAll(java.util.Collection<? extends E>): It is used to add all the elements of a collection into another collection.

```
import java.util.ArrayList;
public class TestAddAll {
public static void main(String[] args) {
        ArrayList a = new ArrayList();
        a.add(20);
        a.add("java by Jagannath");
        a.add(8.9);
        a.add(true);
        System.out.println("Elements in a: "+a);
        ArrayList b = new ArrayList();
        b.add(200);
        b.add("Adv java by Jagannath");
        b.add(89.12);
        b.add('A');
        System.out.println("Elements in b: "+b);
        //add all the elements of a into b
        b.addAll(a);
        System.out.println("Elements in a: "+a);
        System.out.println("Elements in b: "+b);

}
}
```

public abstract boolean removeAll(java.util.Collection<?>): It is used to remove all the elements of collection from another collection if another collection contains the other collection.

e.g:

```
import java.util.ArrayList;
public class TestRemoveAll {
public static void main(String[] args) {
        ArrayList a = new ArrayList();
        a.add(20);
        a.add("java by Jagannath");
        a.add(8.9);
        a.add(true);
        System.out.println("Elements in a: "+a);
        ArrayList b = new ArrayList();
        b.add(200);
        b.add("Adv java by Jagannath");
        b.add(89.12);
        b.add('A');
        System.out.println("Elements in b: "+b);
        //add all the elements of a into b
        b.addAll(a);
        System.out.println("Elements in a: "+a);
        System.out.println("Elements in b: "+b);
        b.removeAll(a);
        System.out.println("Elements in a: "+a);
        System.out.println("Elements in b: "+b);
}
}
```

public abstract boolean retainAll(java.util.Collection<?>): It is used to keep the common elements among the collections safe and remove uncommon elements.

```
import java.util.ArrayList;
public class TestRetainAll {
public static void main(String[] args) throws InterruptedException {
        ArrayList<Integer> a = new ArrayList<Integer>();
        a.add(100);
        a.add(200);
        a.add(300);
        a.add(400);
        ArrayList<Integer> b = new ArrayList<Integer>();
        b.add(10);
        b.add(200);
        b.add(300);
        b.add(40);
        System.out.println("Elements in a: "+a);//100 200 300 400
        System.out.println("Elements in b: "+b);//10 200 300 40
        System.out.println("Adding a into b");
        b.addAll(a);
        Thread.sleep(5000);
        System.out.println("Elements in a: "+a);//100 200 300 400
        System.out.println("Elements in b: "+b);//10 200 300 40 100 200 300 400
        System.out.println("Retaining elements in b: ");
        b.retainAll(a);
        Thread.sleep(10000);
```

```
        System.out.println("Elements in a: "+a);//100 200 300 400
        System.out.println("Elements in b: "+b);
}
}
```

public abstract void clear(): It is used to remove all the elements from the container objects.

```
package com.nit.collectionprograms;
import java.util.ArrayList;
public class TestClear {
        public static void main(String[] args) {
                ArrayList<String> fruit = new ArrayList<String>();
                System.out.println("Adding elements into arraylist");
                fruit.add("apple");
                fruit.add("grapes");
                fruit.add("orange");
                //fruit.add(200);error as only String is allowed because of type specification
                System.out.println("Elements in arraylist : ");
                System.out.println(fruit);
                //Remove all the elements from the arraylist
                fruit.clear();
                System.out.println("After removing all elements: ");
                System.out.println("Array list empty: "+fruit.isEmpty());
                System.out.println("Size: "+fruit.size());
                System.out.println("Elements in arraylist: "+fruit);
        }
}
 public abstract boolean equals(java.lang.Object);
 public abstract int hashCode();
```

//WAP to remove even numbers from a collection and display them.

```
public class TestRemoveIf {
        public static void removeEvenNumbers(ArrayList<Integer> num) {
                for(int i=0;i<num.size();i++) {
                //retrieving each element from collection based on index using get(int index)
                        if(num.get(i)%2==0) {
                                num.remove(i);
                        }
                }
                System.out.println("Elements after removing even nos: "+num);
        }
public static void main(String[] args) {
        ArrayList<Integer> a = new ArrayList<Integer>();
        a.add(109);
        a.add(26);
        a.add(23);
        a.add(40);
        a.add(245);
        System.out.println("Elements in a: "+a);
        removeEvenNumbers(a);
}
}
```

NOTE: In the above program we have to write a logic to remove elements from a collection which is a long process. If the developer has to remove the elements based other conditions then developer will have to design multiple methods each with logic to remove element based on condition. It's a slow development process hence its recommended to use feature of Java 8 which is available in Collection i.e, **default method** in Collection.

public default boolean removeIf(java.util.function.Predicate<? super E>): It was introduced in Java8 to let the developers remove elements based on the conditions.

```
import java.util.ArrayList;
public class TestRemoveIf {
public static void main(String[] args) {
        ArrayList<Integer> a = new ArrayList<Integer>();
        a.add(109);
        a.add(26);
        a.add(23);
        a.add(40);
        a.add(245);
        System.out.println("Elements in a: "+a);//109 26 23 40 245
        a.removeIf(n->n%2==0);//removing based on even number
        System.out.println("Elements in a after removing even nos: "+a);//109 23 245
        a.removeIf(n-> n%5==0);//removong based on divisible by 5
        System.out.println("Elements in a after removing nos divisible by 5: "+a);//109 23
}
}
```

```
 public default java.util.Spliterator<E> spliterator();
 public default java.util.stream.Stream<E> stream();
 public default java.util.stream.Stream<E> parallelStream();
```

List:

- Its an interface which acts as sub interface of Collection(I).
- It is an ordered collection, means it stores the elements on the basis of index.
- It is an interface which is available in java.util package hence we must import it for its use in java program.
- We use list to allow the developers to store multiple number of elements whether they are homogeneous, heterogeneous, duplicate or unique.
- The user of this interface has precise control over where in the list each element is inserted. The user can access elements by their integer index (position in the list), and search for elements in the list.

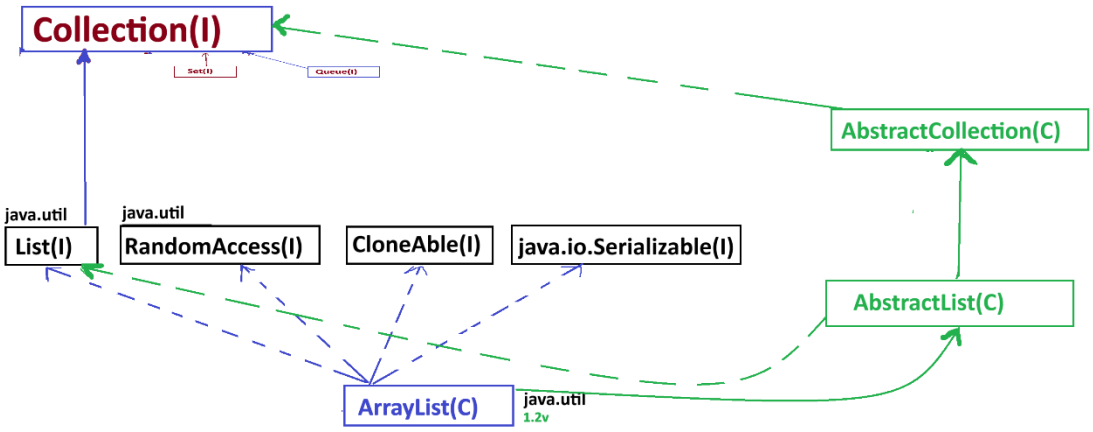
To utilize the functionalities of List(I), we have implementation classes as follows:

i. **ArrayList**
ii. **LinkedList**
iii. **Vector**
Stack (Sub class of Vector)
NOTE: Vector and Stack are available since Java 1.0v that's why we can call them as **Legacy Classes.**

ArrayList(C):

- ArrayList is a predefined class available in java.util.
- ArrayList is considered to implementation class for List(I).
- The underlying data structure of ArrayList is resizable array.
- Default capacity of ArrayList is 10.

- New Capacity of ArrayList is (oldCapacity*3/2) + 1.
- It supports duplicate elements.
- It is heterogeneous in nature.
- It supports null as element.
- Insertion order is preserved.
- It is non thread safe and the operations available in it are non-synchronized.
- It supports serialization.
- It supports cloning.
- If the continuous process is about adding or removing elements from a list, then ArrayList is not a suitable choice.



- To create ArrayList object we have three ways (because CO is performed in AL) :
  a. public java.util.ArrayList(int);
  b. public java.util.ArrayList();
  c. public java.util.ArrayList(java.util.Collection<? extends E>);

//WAP to create an ArrayList object with default capacity(no parameter).

```
import java.util.ArrayList;
public class TestArrayList1 {
public static void main(String[] args) {
        ArrayList a = new ArrayList();//AL object created with default capacity 10
        a.add(100);//PTV  --> IWCO---> O
        a.add(true);//PTV --> BWCO---> O
        a.add("java");// SO ---> O
        a.add(10.8);// PTV --> DWCO --> O
        System.out.println(a);//toString() is invoked
}
}
//Demonstrating copy of another collection in arraylist
package com.nit.collectionprograms;

import java.util.ArrayList;
public class TestArrayList {
public static void main(String[] args) {
        ArrayList<Integer> a1 = new ArrayList<Integer>();
        a1.add(100);
```

```
        a1.add(200);
        a1.add(300);
        a1.add(400);
        ArrayList<Integer> a2 = new ArrayList<Integer>(a1);
        //traversing through elements of a1 using foreach()
                System.out.println("Elements in a1: ");
                a1.forEach(System.out::println);
        //traversing through elements of a2 using foreach()
        System.out.println("Elements in a2: ");
        a2.forEach(System.out::println);
        System.out.println("Changing value in a2 for index 2: "+a2.set(2, 900));
        //Making changes in a2 will affect the a1?NO

        //traversing through elements of a1 using foreach()
                        System.out.println("Elements in a1: ");
                        a1.forEach(System.out::println);
                //traversing through elements of a2 using foreach()
                System.out.println("Elements in a2: ");
                a2.forEach(System.out::println);

        }
}

//ArrayList supports duplicates
import java.util.ArrayList;
public class TestArrayList2 {
public static void main(String[] args) {
        ArrayList a = new ArrayList();
        a.add(100);
        a.add(true);
        a.add("java");
        a.add(80.9);
        a.add(100);//Duplicate allowed
        System.out.println(a);
}
}

//ArrayList supports null
import java.util.ArrayList;
public class TestArrayList2 {
public static void main(String[] args) {
        ArrayList a = new ArrayList();
        a.add(100);
        a.add(true);
        a.add(null);
        a.add("java");
        a.add(80.9);
        a.add(null);
        System.out.println(a);
}
}
```

Retrieving elements from an arraylist.

```
//Program to demonstrate how to traverse through elements in collection
//Program to demonstrate how to traverse through elements in collection
/*
 Traversing of elements is done to retrieve each element at a time.
 To traverse through the elements we have mutiple ways:
    - using get(int index)
    - using for extended loop(for each loop)
    - using Enumeration cursor object
    - using Iterator Cursor object
    - using ListIterator Cursor object
    - using java 8 feature i.e, default method 'forEach()'
 */
import java.util.ArrayList;
class Product{
        private String productName;
        private String productID;
        private int quantity;
        private double price;
        private String category;
        public Product(String productName, String productID, int quantity,
                        double price,String category) {
                this.productName = productName;
                this.productID = productID;
                this.quantity = quantity;
                this.price = price;
                this.category = category;
        }
        public String getProductName() {
                return productName;
        }
        public String getProductID() {
                return productID;
        }
        public int getQuantity() {
                return quantity;
        }
        public double getPrice() {
                return price;
        }
        public String getCategory() {
                return category;
        }
        @Override
        public String toString() {
                return "ProductName: " + productName +
                                "\nProductID: " + productID +
                                "\nCategory: "+category +
                                "\nQuantity: " + quantity +
          "\nPrice: Rs. " + price+"\n";
        }
}

public class Amazon {
```

```
    public static void placeOrder(ArrayList<Product> productCart) {
                //retrieving each product from the cart to display using get(int index)
         System.out.println("Products avaialble in your cart: ");
        for(int i=0;i<productCart.size();i++) {
                System.out.println(productCart.get(i));
        }
        }
   public static void main(String[] args) {
        ArrayList<Product> productCart = new ArrayList<Product>();
        Product p1 = new Product("Water Bottle", "WB123", 4, 30, "Home Needs");
        Product p2 = new Product("DairyMilk Bubbly", "DB345", 1, 90, "Sweet Tooth");
        Product p3 = new Product("Moov Spray", "MVS890", 5, 120, "Pharmacy & Wellness");
        //adding product into cart
        productCart.add(p1);
        productCart.add(p2);
        productCart.add(p3);
        placeOrder(productCart);
}
}

//Retrieving each element using for each loop
package com.nit.collectionprograms;
//Program to demonstrate how to traverse through elements in collection
/*
 Traversing of elements is done to retrieve each element at a time.
 To traverse through the elements we have mutiple ways:
    - using get(int index)
    - using for extended loop(for each loop)
    - using Enumeration cursor object
    - using Iterator Cursor object
    - using ListIterator Cursor object
    - using java 8 feature i.e, default method 'forEach()'
 */
import java.util.ArrayList;
class Product{
        private String productName;
        private String productID;
        private int quantity;
        private double price;
        private String category;
        public Product(String productName, String productID, int quantity,
                        double price,String category) {
                this.productName = productName;
                this.productID = productID;
                this.quantity = quantity;
                this.price = price;
                this.category = category;
        }
        public String getProductName() {
                return productName;
        }
        public String getProductID() {
                return productID;
```

```
        }
        public int getQuantity() {
                return quantity;
        }
        public double getPrice() {
                return price;
        }
        public String getCategory() {
                return category;
        }
        @Override
        public String toString() {
                return "ProductName: " + productName +
                                "\nProductID: " + productID +
                                "\nCategory: "+category +
                                "\nQuantity: " + quantity +
          "\nPrice: Rs. " + price+"\n";
        }
}

public class Amazon {
    public static void placeOrder(ArrayList<Product> productCart) {
                //retrieving each product from the cart to display using for each loop
         System.out.println("Products avaialble in your cart: ");
        for(Product p:productCart) {
                System.out.println(p);
        }
        }
   public static void main(String[] args) {
        ArrayList<Product> productCart = new ArrayList<Product>();
        Product p1 = new Product("Water Bottle", "WB123", 4, 30, "Home Needs");
        Product p2 = new Product("DairyMilk Bubbly", "DB345", 1, 90, "Sweet Tooth");
        Product p3 = new Product("Moov Spray", "MVS890", 5, 120, "Pharmacy & Wellness");
        //adding product into cart
        productCart.add(p1);
        productCart.add(p2);
        productCart.add(p3);
        placeOrder(productCart);
}
}
```


Retrieving elements using Enumeration

```
//Program to demonstrate how to traverse through elements in collection
/*
 Traversing of elements is done to retrieve each element at a time.
 To traverse through the elemenets we have mutiple ways:
    - using get(int index)
    - using for extended loop(for each loop)
    - using Enumeration cursor object
    - using Iterator Cursor object
    - using ListIterator Cursor object
    - using java 8 feature i.e, default method 'forEach()'
 */
```

```
import java.util.ArrayList;
import java.util.Enumeration;
import java.util.Vector;
class Product{
	private String productName;
	private String productID;
	private int quantity;
	private double price;
	private String category;
	public Product(String productName, String productID, int quantity,
			double price,String category) {
		this.productName = productName;
		this.productID = productID;
		this.quantity = quantity;
		this.price = price;
		this.category = category;
	}
	public String getProductName() {
		return productName;
	}
	public String getProductID() {
		return productID;
	}
	public int getQuantity() {
		return quantity;
	}
	public double getPrice() {
		return price;
	}
	public String getCategory() {
		return category;
	}
	@Override
	public String toString() {
		return "ProductName: " + productName +
				"\nProductID: " + productID +
				"\nCategory: "+category +
				"\nQuantity: " + quantity +
		 "\nPrice: Rs. " + price+"\n";
	}
}
public class Amazon {
   public static void placeOrder(ArrayList<Product> productCart) {
		//retrieving each product from the cart to display using Enumeration
	 System.out.println("Products avaialble in your cart: ");
	 //To demonstrate use of enumeration we need Legacy class
	 //Creating Vector from an ArrayList
	 Vector<Product> v = new Vector<Product>(productCart);
	Enumeration<Product> p = v.elements();
	while (p.hasMoreElements()) {
			Product product = p.nextElement();
			System.out.println(product);
		}
```

```
        }
    public static void main(String[] args) {
        ArrayList<Product> productCart = new ArrayList<Product>();
        Product p1 = new Product("Water Bottle", "WB123", 4, 30, "Home Needs");
        Product p2 = new Product("DairyMilk Bubbly", "DB345", 1, 90, "Sweet Tooth");
        Product p3 = new Product("Moov Spray", "MVS890", 5, 120, "Pharmacy & Wellness");
        //adding product into cart
        productCart.add(p1);
        productCart.add(p2);
        productCart.add(p3);
        placeOrder(productCart);
}
}
```

//Retrieving elements using Iterator:

```
import java.util.ArrayList;
import java.util.Iterator;
public class Flipkart {
        public static void viewCart(ArrayList<Product> a) {
                //Traverse through the elements using Iterator
                Iterator<Product> itr = a.iterator();//return iterator over collection
                while (itr.hasNext()) {
                        Product p = itr.next();//next() used to retrieve element
                        System.out.println(p);
                }

        }
public static void main(String[] args) {
        Product p1 = new Product("Oneplus13", "OP908", 1, 30000, "Mobile");
        Product p2 = new Product("Microphone", "MP753", 1, 7500, "Electronics");
        Product p3 = new Product("Tripod", "TR341", 1, 3000, "Electronics");
        Product p4 = new Product("Laptop Cooler", "LP987", 1, 1500, "Electronics");
        ArrayList<Product> a = new ArrayList<Product>();
        a.add(p1);
        a.add(p2);
        a.add(p3);
        a.add(p4);
        viewCart(a);

}
}
```

- **ArrayList can be created in three ways:**

  1.public java.util.ArrayList():

  The above constructor gets executed when we create arraylist object with no arguments passed to it. Then, an arraylist with default capacity 10 is generated. When we dont know about the size of elements to be stored then we must create arraylist with no arguments.

```
import java.util.ArrayList;
 public class TestArrayList1 {
 public static void main(String[] args) {
        ArrayList a = new ArrayList();//arraylist created with default capacity 10
        System.out.println(a);// []
  }
  }
```

2. public java.util.ArrayList(int);

- Create an arraylist with the given capacity.

```
import java.util.ArrayList;
public class TestArrayList1 {
public static void main(String[] args) {
        ArrayList a = new ArrayList(12);//arraylist created with capacity 12
        System.out.println(a);// []
}
}
```

- It is generally used when we expect the exact number of elements to be stored in arraylist before it resizes.It helps to avoid overhead of resizing the arraylist.

3. public java.util.ArrayList(java.util.Collection<? extends E>);

It creates a copy of any collection.

```
import java.util.ArrayList;
import java.util.LinkedList;
public class TestArrayList1 {
        public static void main(String[] args) {
        LinkedList register = new LinkedList();
         register.add("aman");
         register.add("aman");//duplicate
         register.add("suresh");
         register.add("vinod");
         register.add("nova");
         register.add("nova");//duplicate
         System.out.println("Elements in set: "+register);
         ArrayList finalRegister = new ArrayList(register);//copied the LinkedList to arraylist
         System.out.println("Final register: "+finalRegister);
 }
}
```

LinkedList:

- It is a predefined class available in java.util.
- It extends AbstractSequentialList<E>
  implements List<E>, Deque<E>, Cloneable, java.io.Serializable
- Its underlying data structure is rearranging the nodes.
- It is heterogeneous.
- It allows duplicates.
- It allows null.

- No default capacity.
- It is a good choice to be taken when continuous process is about adding and removing element but memory wise it is bad as compared to ArrayList.
- It supports serialization and cloning.

There are two ways to create a LinkedList:

public java.util.LinkedList();
public java.util.LinkedList(java.util.Collection<? extends E>);

Vector:

- It is a predefined class available in java.util .
- It extends AbstractList<E> implements List<E>, RandomAccess, Cloneable, java.io.Serializable
- It is available since first version of java hence it is also called as legacy class.
- It has dynamic array structure.
- Its default capacity is 10.
- Its new capacity is OldCapacity x 2.
- It is heterogeneous.
- It allows duplicate.
- It allows null.
- It is thread safe.
- It has synchronized operations.
- It supports serialization.
- It supports cloning.

We create vector objects in the following ways:

1. **public java.util.Vector(int, int);**
2. **public java.util.Vector(int);**
3. **public java.util.Vector():** A vector object is created with the default capacity 10. If the vector will get a new capacity here then the new capacity will be OC*2.

e.g:

```
import java.util.Vector;
public class TestVector1 {
public static void main(String[] args) {
        //Creation of vector object using non parameterized constructor
        Vector v = new Vector();
        System.out.println("Capacity of vector before adding elements: "+v.capacity());
        v.add(100);
        v.add(200);
        v.add(500);
        v.add(400);
        v.add(300);
        System.out.println("Elements in vector: "+v);
        System.out.println("Capacity of vector after adding elements: "+v.capacity());
}
}
```

4. **public java.util.Vector(java.util.Collection<? extends E>);**

Constructor	Description
Vector(int, int)	Set initial capacity and custom increment size

Constructor	Description
Vector(int)	Set initial capacity, doubles when full
Vector()	Default capacity = 10, doubles when full
Vector(Collection<? extends E>)	Copy from any other collection

1. Vector(int initialCapacity, int capacityIncrement)
Creates a vector with:
initialCapacity: Starting capacity
capacityIncrement: How much capacity increases when full

e.g:
```
import java.util.Vector;
public class Example1 {
   public static void main(String[] args) {
      Vector<Integer> v = new Vector<>(3, 2); // initial capacity 3, increment by 2
      v.add(1);
      v.add(2);
      v.add(3);
      System.out.println("Before Capacity Increase: " + v.capacity()); // 3
      v.add(4); // Triggers capacity increase
      System.out.println("After Capacity Increase: " + v.capacity()); // 5 (3+2)
   }
}
```

2. Vector(int initialCapacity)
- Creates a vector with given initial capacity.
- When full, capacity doubles automatically.

e.g:
```
import java.util.Vector;
public class Example2 {
   public static void main(String[] args) {
      Vector<String> v = new Vector<>(2);
      v.add("A");
      v.add("B");
      System.out.println("Before Doubling: " + v.capacity()); // 2
      v.add("C"); // Exceeds initial capacity
      System.out.println("After Doubling: " + v.capacity()); // 4
   }
}
```

3. Vector()
- Default constructor.
- Initial capacity = 10
- When full, capacity doubles

e.g:
```
import java.util.Vector;
public class Example3 {
   public static void main(String[] args) {
      Vector<Integer> v = new Vector<>();
      System.out.println("Initial Capacity: " + v.capacity()); // 10
```

```
        for (int i = 1; i <= 11; i++) {
            v.add(i); // Add more than 10 elements
        }
        System.out.println("New Capacity After Doubling: " + v.capacity()); // 20
        System.out.println("Elements: " + v);
    }
}
```

4. Vector(Collection<? extends E> c)
Creates a vector containing all elements from another collection.

e.g:

```
import java.util.*;
public class Example4 {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("Apple", "Banana", "Cherry");
        Vector<String> v = new Vector<>(list);
        System.out.println("Vector Elements: " + v); // [Apple, Banana, Cherry]
    }
}
```